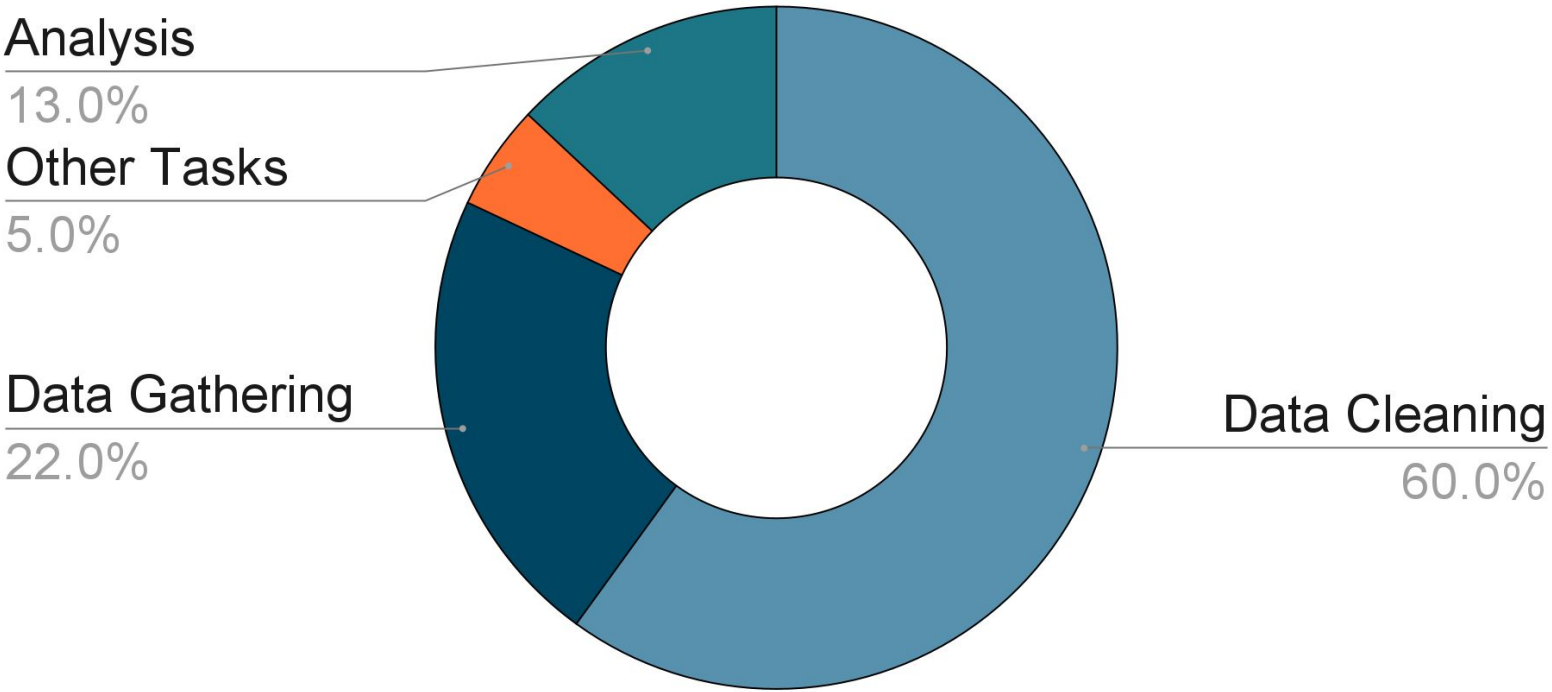
Data Preprocessing and Feature Engineering

Instructor: Caryn Geady, PhD
Date: 19th Jan 2026



Data scientists spend 60% of their time preprocessing data.

https://www.markovml.com/blog/automated-data-cleaning

Why Data Preprocessing Matters

- “Garbage in, garbage out”
  - Poor data ➜ poor models;

- ML algorithms rely on clean, well-prepared data for optimal performance!

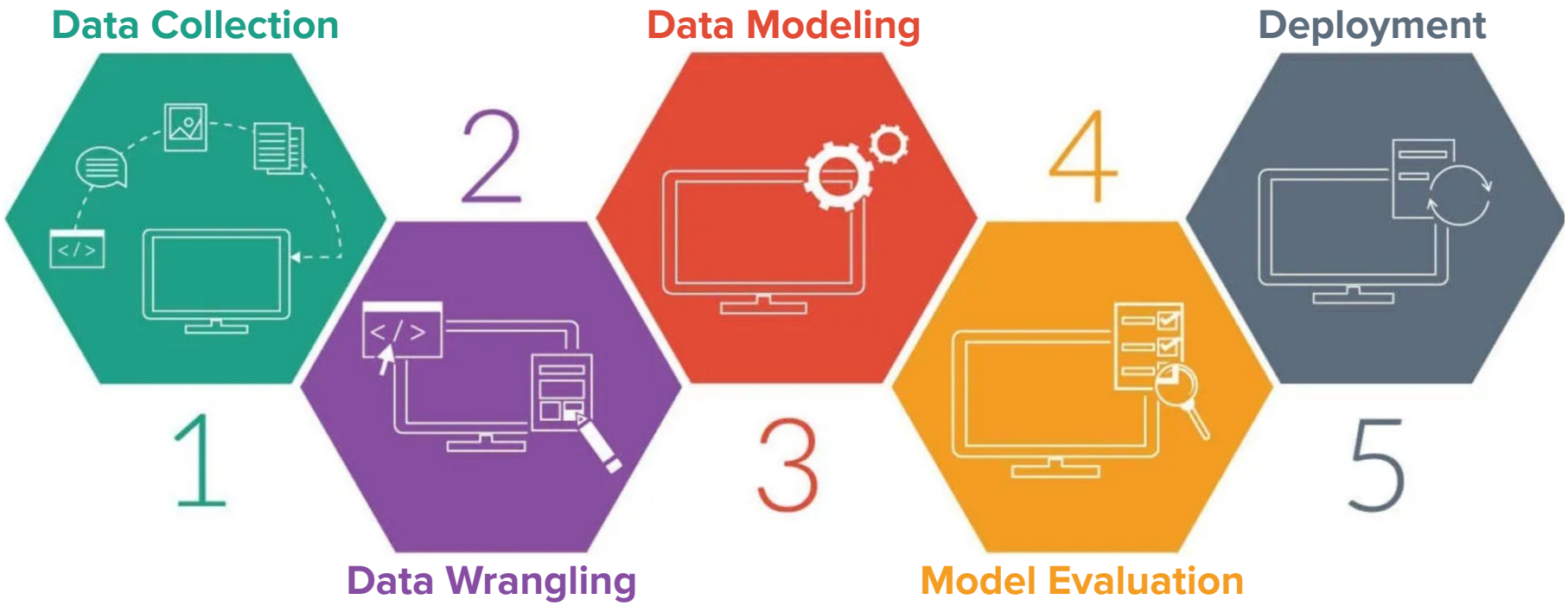
https://towardsdatascience.com/another-twitter-sentiment-analysis-bb5b01ebad90

Today’s Objectives

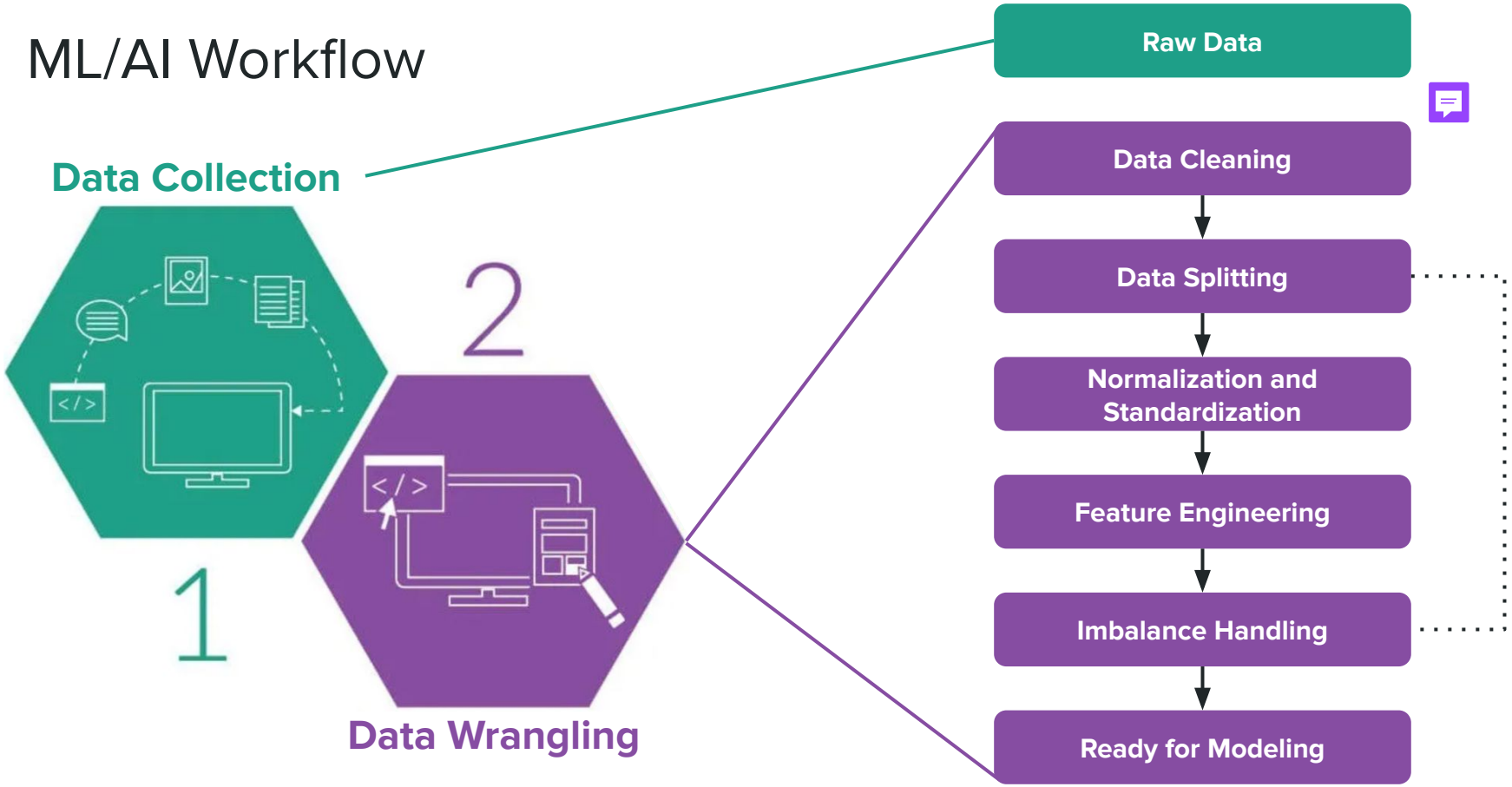
Data Preprocessing Steps:

1. Data Splitting;
2. Data cleaning;
3. Normalization and standardization;
4. Feature selection and engineering;
5. Handling imbalanced data

ML/AI Workflow



ML/AI Workflow



Data Cleaning

Data Cleaning: a Necessary Evil

- Definition: removing noise, fixing errors and ensuring consistency;
- Examples:
  - Handling missing data;
  - Removing duplicates;
  - Fixing inconsistencies.

Practical Approaches to Data Cleaning

Handling Missing Values in Clinical Data

Problem:

- missing age, sex and diagnosis

Solution:

- Fill with appropriate string / numerical data

```
import pandas as pd

# Sample clinical data with missing values
data = {
  'PatientID': [1, 2, 3, 4, 5],
  'Age': [25, 30, None, 35, 40],
  'Sex': ['Male', 'Female', 'Female', None, 'Male'],
  'Diagnosis': ['Diabetes', 'Hypertension', 'Diabetes',
'Hypertension', None]
}
df = pd.DataFrame(data)

# Fill missing values with a specific value or method
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Sex'].fillna('Unknown', inplace=True)
df['Diagnosis'].fillna('Unknown', inplace=True)
```

Practical Approaches to Data Cleaning

Standardizing Biological Sex Labels

Problem:

- Inconsistent sex label

Solution:

- Use a dictionary to map to standardized feature

```
# Sample clinical data with inconsistent sex labels
data = {
  'PatientID': [1, 2, 3, 4, 5],
  'Sex': ['M', 'Female', 'F', 'male', 'f']
}
df = pd.DataFrame(data)

# Standardize gender labels
sex_mapping = {'M': 'Male',
       'F': 'Female',
       'male': 'Male',
       'female': 'Female',
       'f': 'Female'
}
df['Sex'] = df['Sex'].map(sex_mapping)
```

Practical Approaches to Data Cleaning

Normalizing Gene Names

Problem:

- Inconsistent gene name

Solution:

- Use string methods for consistent naming

```
# Sample clinical data with inconsistent gene names
data = {
  'PatientID': [1, 2, 3, 4, 5],
  'Gene': ['BRCA1', 'brca1', 'BRCA2', 'brca2',
'BRCA1']
}
df = pd.DataFrame(data)

# Normalize gene names
df['Gene'] = df['Gene'].str.upper()
```

Practical Approaches to Data Cleaning

Converting Clinical Data Types

Problem:

- Numerical data in string format

Solution:

- Pandas `astype()` to convert data accordingly

```
# Sample clinical data with incorrect data types
data = {
  'PatientID': [1, 2, 3, 4, 5],
  'Age': ['25', '30', '35', '40', '45'],
  'BloodPressure': ['120', '130', '140', '150',
'160']
}
df = pd.DataFrame(data)

# Convert data types
df['Age'] = df['Age'].astype(int)
df['BloodPressure'].astype(int)
```

Practical Approaches to Data Cleaning

Handling Outliers in Clinical Measurement

Problem:

- Large outliers present in the data

Solution:

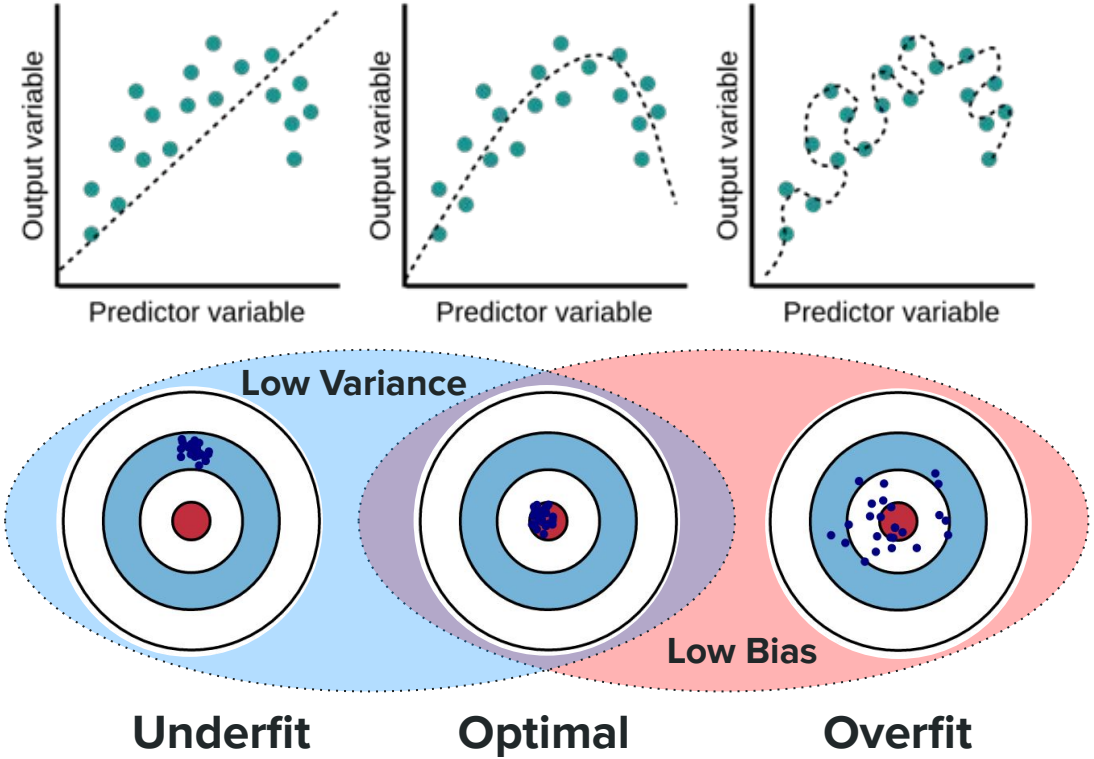
- Statistically-based outlier removal

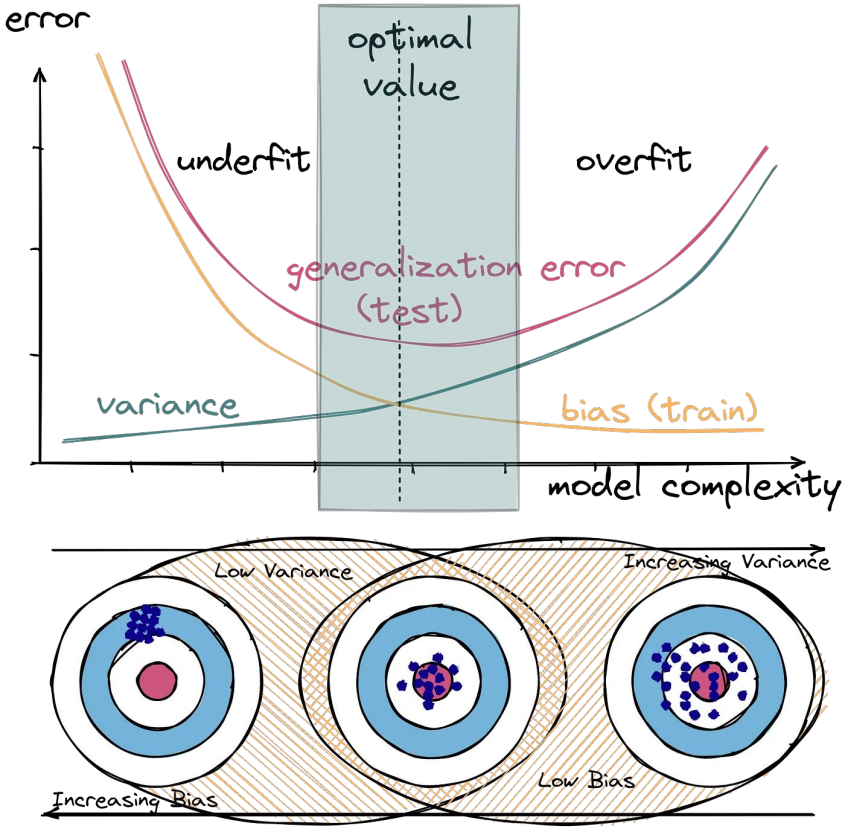
```python
import numpy as np
from scipy import stats

# Sample clinical data with outliers
data = {
  'PatientID': [1, 2, 3, 4, 5],
  'BloodPressure': [120, 130, 140, 150, 300],  # 300 is an outlier
  'Cholesterol': [200, 210, 220, 230, 1000]  # 1000 is an outlier
}
df = pd.DataFrame(data)

# Remove outliers using the Z-score method
z_scores = np.abs(stats.zscore(df[['BloodPressure', 'Cholesterol']]))
df_no_outliers = df[(z_scores < 3).all(axis=1)]
```

Data Splitting

The Importance of Splitting Your Data

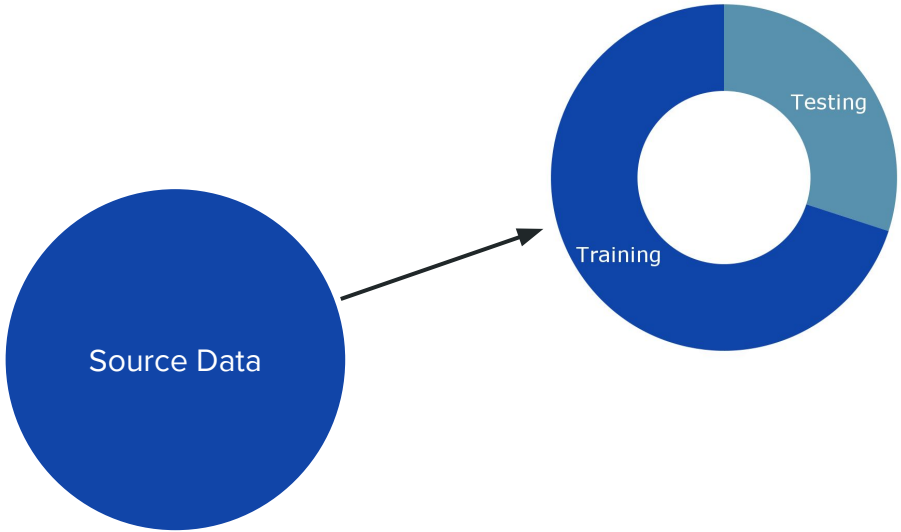
- Data splitting ensure models are evaluated on data they have not seen before;
- Goal ➔ strike a balance between complexity and simplicity:
  - Overfitting (too complex) memorizes training data but fails to generalize;
  - Underfitting (too simple) fails to capture critical patterns in the data




error
optimal value
underfit
overfit
generalization error (test)
variance
bias (train)
model complexity
Low Variance
Increasing Variance
Increasing Bias
Low Bias

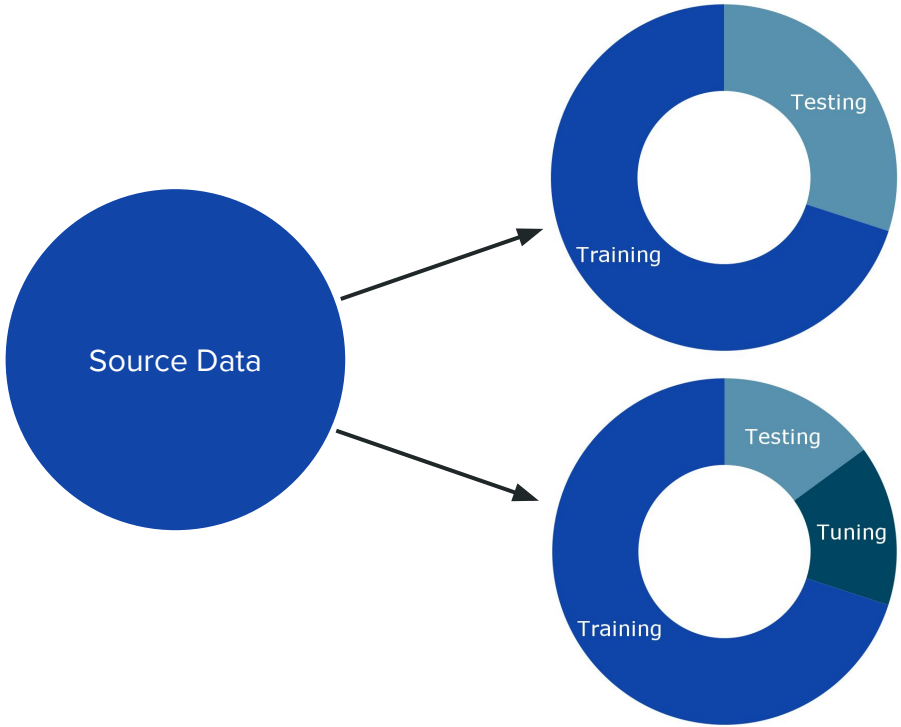
Training vs. Testing Data

- ML models need to generalize well to unseen data;



Training vs. Testing Data (vs. Tuning Data)

- ML models need to generalize well to unseen data;
- Training (to teach), tuning (to fine-tune) and testing (to verify);
- Common ratios:
  - 70/15/15 (smaller datasets);
  - 80/10/10 (larger datasets)



Splitting Ratios and Approaches

Using `sklearn`'s `train_test_split`

- `test_size` denotes the fraction of the dataset reserved for testing;
- Set the random_state input argument for reproducibility!

```
from sklearn.model_selection import train_test_split
import pandas as pd

# Sample data
data = {
  'PatientID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
  'Age': [25, 30, 35, 40, 45, 50, 55, 60, 65, 70],
  'Gender': ['Male', 'Female', 'Female', 'Male', 'Male', 'Female', 'Female',
'Male', 'Male', 'Female'],
  'Diagnosis': ['Diabetes', 'Hypertension', 'Diabetes', 'Hypertension',
'Diabetes', 'Hypertension', 'Diabetes', 'Hypertension', 'Diabetes',
'Hypertension']
}
df = pd.DataFrame(data)

# Split data into training and testing sets
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
```

Splitting Ratios and Approaches

Using a Stratified Split based on a Feature or Target

- Stratify on a given column of your dataframe;
- Important for imbalanced datasets!

```
from sklearn.model_selection import train_test_split
import pandas as pd

# Sample data
data = {
  'PatientID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
  'Age': [25, 30, 35, 40, 45, 50, 55, 60, 65, 70],
  'Gender': ['Male', 'Female', 'Female', 'Male', 'Male', 'Female', 'Female',
'Male', 'Male', 'Female'],
  'Diagnosis': ['Diabetes', 'Hypertension', 'Hypertension', 'Hypertension',
'Diabetes', 'Hypertension', 'Hypertension', 'Hypertension', 'Hypertension',
'Hypertension']
}
df = pd.DataFrame(data)

# Stratified split based on the 'Diagnosis' column
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42,
stratify=df['Diagnosis'])
```

Splitting Ratios and Approaches

Splitting based on Treatment Data

- Useful for time-series data, i.e., when you want to ensure that the training data precedes the testing data.

```
# Sample data with treatment dates
data = {
  'PatientID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
  'Age': [25, 30, 35, 40, 45, 50, 55, 60, 65, 70],
  'Diagnosis': ['Diabetes', 'Hypertension', 'Diabetes', 'Hypertension',
'Diabetes', 'Hypertension', 'Diabetes', 'Hypertension', 'Diabetes',
'Hypertension'],
  'TreatmentDate': pd.to_datetime(['2021-01-01', '2021-02-01', '2021-03-01',
'2021-04-01', '2021-05-01', '2021-06-01', '2021-07-01', '2021-08-01',
'2021-09-01', '2021-10-01'])
}
df = pd.DataFrame(data)

# Split data based on a specific treatment date
split_date = pd.to_datetime('2021-06-01')
train_df = df[df['TreatmentDate'] < split_date]
test_df = df[df['TreatmentDate'] >= split_date]
```

Normalization and Standardization

What are Normalization and Standardization?

(Min-Max) Normalization

- Scales data to fit within a specific range, typically [0, 1]:

$$x' = x - \frac{\mathbf{min}(x)}{\mathbf{max}(x) - \mathbf{min}(x)}$$

- Useful when:
  - Your data **does not follow a normal distribution**;
  - When using **distance-based algorithms** (e.g., k-Nearest Neighbours, k-Means Clustering);
  - When the **scale of the features varies greatly**;
  - When the features are inherently **bounded** (e.g., percentages, image pixel intensities)
- Common use cases:
  - Neural Networks (especially input layers, where normalization helps gradient descent converge faster)

What are Normalization and Standardization?

Standardization

- Transforms the data to have zero-mean and unit variance:

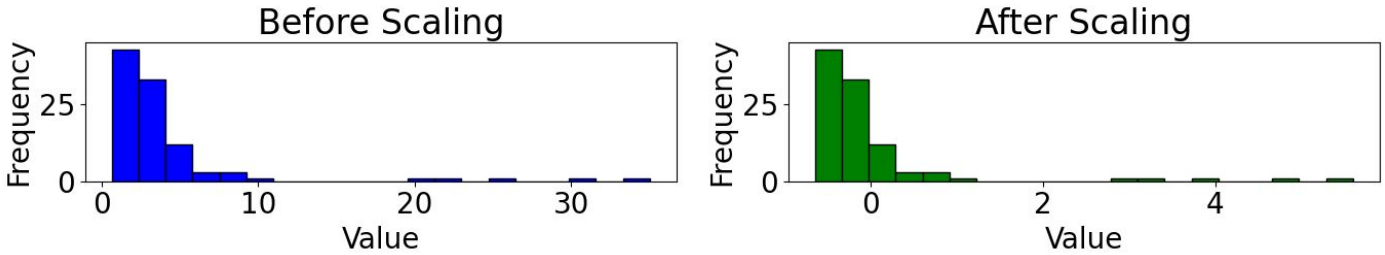
$$x' = \frac{x - \overline{x}}{\sigma}$$

- Useful when:
  - Your data follows a **normal** distribution;
  - When using algorithm **sensitive to the distribution of the data** (e.g., Support Vector Machines (SVM), Principal Component Analysis (PCA), Logistic Regression);
  - When dealing with features with **outliers**;
- Common use cases:
  - Algorithms assuming normality (PCA);
  - Linear models like logistic regression and SVM.

Techniques and Tools

- sklearn's
  - MinMaxScaler
  - StandardScaler
- Before and after scaling:

```python
import numpy as np
from sklearn.preprocessing import StandardScaler

# Sample data with a non-normal (skewed) distribution and extreme outliers
data = np.random.lognormal(mean=1, sigma=0.5, size=95)
outliers = np.array([20, 22, 25, 30, 35])
data = np.concatenate([data, outliers])

# Scaling the data using StandardScaler
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data.reshape(-1, 1)).flatten()
```



Feature Selection and Engineering

Why Feature Selection/Engineering is Crucial

1. **Avoiding the “curse of dimensionality”;**
    - As the number of features increases, the data points become sparse in the feature space, making it difficult for models to generalize well;
    - Impact:
        - Increased computational cost;
        - Risk of overfitting;
        - Reduced interpretability
2. Improving model performance;
3. Handling multicollinearity;
4. Enabling faster training and deployment;
5. Enhancing domain insights.

Why Feature Selection/Engineering is Crucial

1. Avoiding the “curse of dimensionality”;
2. **Improving model performance;**
    - Selecting the most relevant features can:
        - Enhance predictive accuracy;
        - Reduce model complexity;
        - Improve interpretability
3. Handling multicollinearity;
4. Enabling faster training and deployment;
5. Enhancing domain insights.

Why Feature Selection/Engineering is Crucial

1. Avoiding the “curse of dimensionality”;
2. Improving model performance;
3. **Handling multicollinearity;**
    - When two or more features are highly correlated, they provide redundant information;
    - Impact:
        - Reduces the stability of models like linear regression
4. Enabling faster training and deployment;
5. Enhancing domain insights / model interpretability.

Why Feature Selection/Engineering is Crucial

1. Avoiding the “curse of dimensionality”;
2. Improving model performance;
3. Handling multicollinearity;
4. **Enabling faster training and deployment;**
    - Smaller feature sets:
        - Decrease data storage and processing time;
        - Reduce latency during deployment, especially for real-time applications
5. Enhancing domain insights / model interpretability.

Why Feature Selection/Engineering is Crucial

1. Avoiding the “curse of dimensionality”;
2. Improving model performance;
3. Handling multicollinearity;
4. Enabling faster training and deployment;
5. **Enhancing domain insights / model interpretability.**

- Feature engineering allows us to:
  - Incorporate domain knowledge into a model;
  - Derive meaningful insights from the data.

Feature Engineering Techniques

- Transformation:
  - Scaling
  - Log transformations
- Extraction:
  - PCA
  - t-SNE
- Creation
  - Combining features
  - Binning continuous variables

```python
import pandas as pd

# Sample clinical data
data = {
  'PatientID': [1, 2, 3, 4, 5],
  'Weight_kg': [70, 80, 60, 90, 75],
  'Height_cm': [175, 180, 165, 190, 170]
}
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Calculate BMI and add it as a new column
df['Height_m'] = df['Height_cm'] / 100  # Convert height from cm to m
df['BMI'] = df['Weight_kg'] / (df['Height_m'] ** 2)
```

Feature Selection Techniques

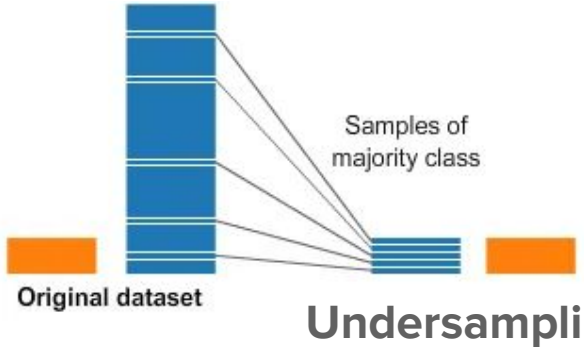
- Filter Methods:
  - Statistical tests (e.g., correlation)
- Algorithmic Methods:
  - Recursive Feature Elimination (RFE)
- Embedded Methods:
  - Lasso regression
  - Decision trees

```
import pandas as pd
import numpy as np

# Sample data with highly correlated features
np.random.seed(42)
data = {
  'Feature_1': np.random.rand(100),
  'Feature_2': np.random.rand(100),
  'Feature_3': np.random.rand(100) * 10,
  'Feature_4': np.random.rand(100) * 10,
  'Feature_5': np.random.rand(100) * 10
}

# Introduce high correlation between Feature_3 and Feature_4
data['Feature_4'] = data['Feature_3'] * 0.95 + np.random.rand(100)
* 0.1

df = pd.DataFrame(data)
```

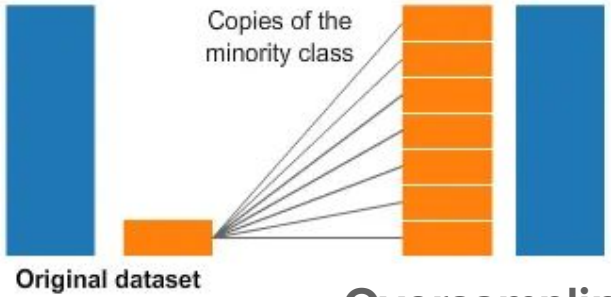
Handling Imbalanced Data

What is Imbalanced Data?

- When one class significantly outnumbers the other(s);
- Examples: fraud detection, rare disease prediction;
- Issues: skewed performance metrics;
- Solutions:
  - Sampling techniques
  - Algorithmic adjustments



Undersampling



Oversampling

Summary and Q&A

Summary

- Data preprocessing ensures:
  - Clean and split datasets;
  - Proper scaling for algorithms;
  - Thoughtful feature engineering;
  - Balanced datasets for robust performance.